

Conception et Déploiement d'un Module d'Entreprise (Pattern Public / Private)

1. Cadrage Opérationnel

L'objectif de cet atelier est d'abandonner définitivement les scripts autonomes (.ps1) dispersés pour concevoir un **module d'administration d'infrastructure standardisé** nommé **InfraTools**.

Ce module devra répondre aux exigences d'ingénierie logicielle en entreprise :

- Découpage strict selon le pattern « **une fonction = un fichier** ».
- Séparation étanche entre les fonctions exposées (**Public/**) et les fonctions d'aide internes masquées (**Private/**).
- Orchestration dynamique via un chargeur racine (.psm1) par *dot-sourcing*.
- Génération d'un manifeste certifié (.psd1) avec énumération explicite des fonctions exportées.
- Déploiement dans l'arborescence officielle pour valider le mécanisme d'auto-chargement (*Auto-loading*).

2. Architecture Cible du Module

Vous devez construire l'arborescence suivante dans votre espace de travail :

```
InfraTools/  
├── InfraTools.psd1      # Manifeste du module (identité, version, exports)  
├── InfraTools.psm1      # Chargeur dynamique du module  
├── Public/              # Fonctions métiers exposées aux administrateurs  
│   ├── Get-InfraDisque.ps1  
│   └── Test-FluxPort.ps1  
└── Private/             # Fonctions utilitaires internes (invisibles)  
    ├── Write-JournalModule.ps1  
    └── Convert-TailleOctet.ps1
```

3. Cahier des Charges Étape par Étape

Étape 1 : Développement des Fonctions Privées (Private/)

Ces fonctions accomplissent des tâches techniques répétitives. Elles ne doivent jamais être appelées directement par l'utilisateur final.

1. Private\Convert-TailleOctet.ps1

- **Rôle** : Convertir une taille brute en octets vers une unité lisible en gigaoctets (Go) ou

mégaoctets (Mo) avec un arrondi précis.

- **Spécifications :**
 - Accepter un paramètre obligatoire [long]\$Octets.
 - Accepter un paramètre optionnel [string]\$Unite restreint à "GB" et "MB" via [ValidateSet()] (valeur par défaut : "GB").
 - Retourner un nombre à virgule ([double]) arrondi à une décimale via [math]::Round().
 - Renvoyer uniquement la valeur calculée (aucun message texte en sortie).

2. Private\Write-JournalModule.ps1

- **Rôle :** Écrire un message d'audit horodaté dans un fichier journal centralisé sans perturber le flux de données standard.
- **Spécifications :**
 - Accepter un paramètre obligatoire [string]\$Message.
 - Accepter un paramètre optionnel [string]\$Niveau restreint à "INFO", "WARNING", "ERROR" via [ValidateSet()] (valeur par défaut : "INFO").
 - Chemin cible du journal : C:\Audit\infratools.log.
 - Créer le répertoire parent s'il est absent.
 - Inscrire la ligne horodatée au format [yyyy-MM-dd HH:mm:ss] [NIVEAU] Message avec encodage strict UTF-8.
 - Émettre un message informatif sur le canal verbeux (Write-Verbose) pour faciliter le débogage.

Étape 2 : Développement des Fonctions Publiques (Public/)

Ces fonctions constituent les commandes officielles du module. Chaque fichier doit porter **strictement le même nom** que la fonction qu'il contient.

1. Public\Get-InfraDisque.ps1

- **Rôle :** Auditer l'espace disque des partitions locales fixes et renvoyer un objet structuré.
- **Spécifications :**
 - Déclarer [CmdletBinding()].
 - Rédiger un bloc d'aide complet en tête de fonction (.SYNOPSIS, .DESCRIPTION, .PARAMETER, .EXAMPLE).
 - Accepter un paramètre obligatoire [string[]]\$Lecteur acceptant les entrées depuis le pipeline (ValueFromPipeline = \$true).
 - Accepter un paramètre [int]\$SeuilAlertePct = 15.
 - Utiliser la cmdlet CIM adéquate pour interroger les disques fixes locaux (DriveType = 3) avec filtrage WQL à la source.
 - Appeler la fonction privée Convert-TailleOctet pour transformer les tailles en gigaoctets.
 - Calculer le pourcentage d'espace libre restant.
 - Si le pourcentage passe sous le seuil, consigner une ligne d'avertissement via Write-JournalModule.
 - **Sortie :** Renvoyer exclusivement un objet [PSCustomObject] pour chaque partition

avec les propriétés : Lecteur, TailleGo, LibreGo, LibrePct, Statut ("OPTIMAL" ou "CRITIQUE").

- **Interdiction formelle** : Ne jamais appeler Write-Host, Format-Table ou Out-File à l'intérieur de la fonction.

2. Public\Test-FluxPort.ps1

- **Rôle** : Tester l'accessibilité d'un port TCP sur une ou plusieurs cibles réseau.
- **Spécifications** :
 - Déclarer [CmdletBinding()].
 - Rédiger le bloc de documentation d'aide intégrée (Comment-Based Help).
 - Accepter un paramètre obligatoire [string[]]\$Cibles (supportant le pipeline).
 - Accepter un paramètre obligatoire [int]\$Port borné entre 1 et 65 535 via [ValidateRange()].
 - Tester l'ouverture du socket TCP sur chaque machine (via Test-NetConnection ou l'objet .NET System.Net.Sockets.TcpClient).
 - Tracer le résultat dans le journal via Write-JournalModule.
 - **Sortie** : Renvoyer un [PSCustomObject] contenant : Cible, Port, Ouvert (booléen \$true ou \$false), DateControle.

Étape 3 : Programmation du Script Chargeur (InfraTools.psm1)

Le fichier .psm1 ne doit contenir aucune logique métier. Il agit comme chef d'orchestre de l'arborescence :

1. Résoudre le chemin absolu du dossier du module via la variable automatique \$PSScriptRoot.
2. Parcourir tous les fichiers .ps1 présents dans le sous-dossier Private/ et les importer dans la session du module via l'opérateur de *dot-sourcing* (.).
3. Parcourir tous les fichiers .ps1 présents dans le sous-dossier Public/ et les importer de la même manière.
4. Déclarer l'instruction **Export-ModuleMember** en ciblant **exclusivement** les noms de base des fonctions issues de Public/.
5. Vérifier qu'aucun alias, variable ou fonction privée n'est mentionné dans l'export.

Étape 4 : Génération et Configuration du Manifeste (InfraTools.psd1)

On ne crée pas de manifeste à la main dans un fichier texte vierge.

1. Utilisez la commande native **New-ModuleManifest** pour initialiser InfraTools.psd1 avec les métadonnées suivantes :
 - Chemin du manifeste : .\InfraTools\InfraTools.psd1
 - Script racine (RootModule) : 'InfraTools.psm1'
 - Version sémantique (ModuleVersion) : '1.0.0'
 - Auteur : Votre nom ou votre équipe
 - Description : Description opérationnelle du module

- Version PowerShell minimale requise : '5.1'
 - Fonctions à exporter (FunctionsToExport) : Liste explicite des deux fonctions publiques ('Get-InfraDisque', 'Test-FluxPort')
 - Désactivation des exports superflus : fixer CmdletsToExport, VariablesToExport et AliasesToExport sur un tableau vide @().
2. Ouvrez le fichier .psd1 généré et vérifiez l'absence de caractères génériques (*) sur la clé FunctionsToExport.

Étape 5 : Déploiement dans \$env:PSModulePath

Pour que PowerShell détecte automatiquement le module sans chemin relatif :

1. Affichez les chemins référencés par la variable d'environnement :
`$env:PSModulePath -split ';'`
2. Choisissez l'emplacement machine partagé : C:\Program Files\WindowsPowerShell\Modules
3. Créez un sous-dossier portant **exactement le même nom** que le module :
C:\Program Files\WindowsPowerShell\Modules\InfraTools\
4. Copiez l'intégralité du dossier InfraTools (avec ses sous-dossiers Public/ et Private/, ainsi que les fichiers .psm1 et .psd1) dans cet emplacement.

4. Protocole de Validation et Recette Opérationnelle

Suivez rigoureusement les 5 tests ci-dessous pour valider la conformité de votre package.

Test 1 : Validation syntaxique du manifeste

Exécutez dans une console d'administration :

```
Test-ModuleManifest -Path "C:\Program
Files\WindowsPowerShell\Modules\InfraTools\InfraTools.psd1"
```

- **Résultat obligatoire** : PowerShell affiche la table de description du module. Aucune ligne rouge ne doit apparaître.

Test 2 : Validation de l'Auto-loading en session vierge

1. **Fermez complètement** toutes les fenêtres PowerShell et votre éditeur.
2. Ouvrez une nouvelle session PowerShell classique (sans droits élevés).
3. Ne tapez **aucun** Import-Module.
4. Tapez la commande suivante :
`Get-Command -Module InfraTools`

- **Résultat obligatoire** : Les deux fonctions Get-InfraDisque et Test-FluxPort apparaissent

immédiatement dans la liste.

Test 3 : Audit d'isolation et d'encapsulation (Sécurité)

Dans la même session, vérifiez que les fonctions internes restent strictement inaccessibles :

Vérification des fonctions privées

Get-Command Write-JournalModule -ErrorAction SilentlyContinue

Get-Command Convert-TailleOctet -ErrorAction SilentlyContinue

- **Résultat obligatoire :** Ces commandes ne retournent rien (\$null). Tenter d'exécuter Convert-TailleOctet déclenche une erreur indiquant que la commande n'est pas reconnue.

Test 4 : Contrôle de l'aide intégrée

Interrogez l'aide de votre fonction :

Get-Help Get-InfraDisque -Detailed

Get-Help Test-FluxPort -Examples

- **Résultat obligatoire :** L'aide affiche le synopsis, le détail de chaque paramètre et les exemples d'utilisation rédigés dans les en-têtes.

Test 5 : Exécution fonctionnelle et pipeline

Validez le fonctionnement en chaîne et la production d'objets purs :

1. Test unitaire du disque local C:

Get-InfraDisque -Lecteur "C"

2. Test via le pipeline vers un filtre et un export

@("C", "D") | Get-InfraDisque -SeuilAlertePct 20 | Where-Object Statut -eq "OPTIMAL"

3. Test de flux réseau

Test-FluxPort -Cibles "127.0.0.1" -Port 5985

- **Résultat obligatoire :**
 - Les données sortent sous forme d'objets exploitables.
 - Le fichier C:\Audit\infratools.log est alimenté proprement en arrière-plan.